perplexity

Show me the full DDL for the housing allocations table

Walk me through the pgTAP assertion suite for isolation tests
Show how the default-deny RLS policies are structured
Draft the LTI 1.3 audit check for token claims
Review the FERPA retention safeguard implementation

The housing allocation DDL below includes its constraints, indexes, version trigger, and default-deny RLS. The isolation suite is a useful baseline, but the retention safeguard needs stronger concurrency and privileged-access controls before production use.

The LTI audit check must record results from real validation—not infer validity by decoding a JWT. None of the code below has been executed against your deployment.

1. Housing allocation DDL

This migration assumes these tables already exist with composite keys on `(tenant_id, id)`:

- `semester.housing_cycles`
- `semester.housing_beds`
- `semester.people`

It is a fresh-table migration, not an upgrade script for an existing `housing_allocations` table.

```
BEGIN;

CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE semester.housing_allocations (
    tenant_id uuid NOT NULL,
    id uuid NOT NULL DEFAULT gen_random_uuid(),

    cycle_id uuid NOT NULL,
    bed_id uuid NOT NULL,
    person_id uuid NOT NULL,

    -- Half-open occupancy interval: [start, end).
    occupancy_range tstzrange NOT NULL,

    state text NOT NULL CHECK (
        state IN (
            'reserved',
            'offered',
```

```
            'contracted',
            'occupied',
            'completed',
            'cancelled',
            'expired'
        )
    ),

    expires_at timestamptz,
    request_key text NOT NULL,

    created_at timestamptz NOT NULL DEFAULT now(),
    updated_at timestamptz NOT NULL DEFAULT now(),
    version bigint NOT NULL DEFAULT 1,

    PRIMARY KEY (tenant_id, id),

    UNIQUE (tenant_id, request_key),

    FOREIGN KEY (tenant_id, cycle_id)
        REFERENCES semester.housing_cycles (tenant_id, id),

    FOREIGN KEY (tenant_id, bed_id)
        REFERENCES semester.housing_beds (tenant_id, id),

    FOREIGN KEY (tenant_id, person_id)
        REFERENCES semester.people (tenant_id, id),

    CHECK (length(request_key) BETWEEN 1 AND 200),

    CHECK (
        NOT isempty(occupancy_range)
        AND NOT lower_inf(occupancy_range)
        AND NOT upper_inf(occupancy_range)
        AND lower_inc(occupancy_range)
        AND NOT upper_inc(occupancy_range)
    ),

    CHECK (
        state <> 'reserved'
        OR expires_at IS NOT NULL
    ),

    CHECK (version >= 1),

    CONSTRAINT housing_bed_overlap
    EXCLUDE USING gist (
        tenant_id WITH =,
        bed_id WITH =,
        occupancy_range WITH &&
    )
    WHERE (
        state IN (
            'reserved',
            'offered',
            'contracted',
```

```
            'occupied'
        )
    ),

    CONSTRAINT housing_person_overlap
    EXCLUDE USING gist (
        tenant_id WITH =,
        person_id WITH =,
        occupancy_range WITH &&
    )
    WHERE (
        state IN (
            'reserved',
            'offered',
            'contracted',
            'occupied'
        )
    )
);

CREATE INDEX housing_allocations_person_idx
ON semester.housing_allocations (
    tenant_id,
    person_id,
    state
);

CREATE INDEX housing_allocations_cycle_idx
ON semester.housing_allocations (
    tenant_id,
    cycle_id,
    state
);

CREATE INDEX housing_allocations_expiry_idx
ON semester.housing_allocations (
    tenant_id,
    expires_at
)
WHERE state = 'reserved';


CREATE FUNCTION semester.housing_allocation_version()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = pg_catalog
AS $$
BEGIN
    IF NEW.tenant_id IS DISTINCT FROM OLD.tenant_id THEN
        RAISE EXCEPTION 'Allocation tenant cannot change'
            USING ERRCODE = '23514';
    END IF;

    NEW.created_at := OLD.created_at;
    NEW.updated_at := clock_timestamp();
    NEW.version := OLD.version + 1;
```

```
    RETURN NEW;
END;
$$;

REVOKE ALL ON FUNCTION
    semester.housing_allocation_version()
FROM PUBLIC;

CREATE TRIGGER housing_allocation_version
BEFORE UPDATE ON semester.housing_allocations
FOR EACH ROW
EXECUTE FUNCTION semester.housing_allocation_version();


ALTER TABLE semester.housing_allocations
    ENABLE ROW LEVEL SECURITY;

ALTER TABLE semester.housing_allocations
    FORCE ROW LEVEL SECURITY;

REVOKE ALL ON semester.housing_allocations FROM PUBLIC;

COMMIT;
```

What each safeguard does

Safeguard	Effect
Composite foreign keys	Prevent references to another institution's cycle, bed, or person
Unique request key	Supports idempotent allocation commands
Bounded half-open range	Rejects empty/unbounded intervals; permits back-to-back occupancy
Bed exclusion	Prevents overlapping active claims on the same bed
Person exclusion	Prevents overlapping active residence claims for one person
Expiration index	Supports efficient reservation-expiry processing
Version trigger	Records changes and prevents tenant reassignment
Default-deny RLS	Blocks ordinary row access until approved policies exist

Command-layer requirements

The DDL does not determine housing eligibility or whether an actor can change a state.

The confirmation command must:

1. Authenticate the actor.
2. Verify the selection window and eligibility.
3. Lock the reservation.

4. Check ownership and expiration inside the transaction.
5. Confirm that the requested transition is permitted.
6. Update the allocation using the expected version.
7. Create assignment/contract records as applicable.
8. Persist audit and outbox events.
9. Return a confirmed result.

An expired timestamp does not automatically release the bed. A worker must change the state to `expired`, coordinated with confirmation.

2. Isolation assertion walkthrough

The previously generated suite contains 10 assertions.

Assertion-by-assertion meaning

#	Assertion	What it proves	What it does not prove
1	All tables enable RLS	Row-security enforcement is configured	Policies are correct
2	All tables force RLS	Owners are subject to RLS in ordinary circumstances	Superusers cannot bypass it
3	Tenant-owned foreign keys include tenant ID	Cross-tenant references have a structural boundary	Runtime users cannot read unrelated rows
4	Bed overlap exclusion exists	Allocation constraint is installed	Concurrent allocation behavior passes
5	Probe role lacks superuser/BYPASSRLS	Probe uses a meaningful security boundary	Production roles match that configuration
6	Probe sees no people	Populated identity rows are hidden by default-deny	Authorized users can read their own records
7	Probe sees no housing applications	Baseline query executes without unauthorized visibility	Strong isolation if the table is empty
8	Probe sees no dining enrollments	Same baseline behavior for dining	Strong isolation if the table is empty
9	Unauthorized insert fails with `42501`	Table privileges deny that write	An RLS `WITH CHECK` policy rejects tenant spoofing
10	Cross-tenant reference fails with `23503`	Composite FK rejects the wrong-tenant person	RLS prevents other forms of access

Assertions 7 and 8 need populated fixtures to become meaningful leakage tests. Assertion 9 currently proves privilege denial, not the correctness of a write policy.

Cross-tenant constraint test

```
SELECT throws_ok(
    $sql$
    INSERT INTO semester.billing_charges (
        tenant_id,
        person_id,
        amount_minor,
        currency,
        status
    )
    VALUES (
        '10000000-0000-0000-0000-000000000001',
        '30000000-0000-0000-0000-000000000003',
        100,
        'USD',
        'pending'
    )
    $sql$,
    '23503',
    NULL,
    'Tenant A cannot reference a person belonging only to tenant B'
);
```

This assumes the fixture person exists only in tenant B and setup runs under an isolated fixture administrator.

Strengthen the suite

Add positive and negative tests against the actual production policies:

Test pair	Required result
Student reads own assignment / another student's assignment	Own visible; other denied
A staff reads A records / B records	Assigned A scope visible; B denied
Valid insert / insert with another tenant	Valid succeeds; spoofed tenant rejected
Valid update / update moving record to another tenant	Valid succeeds; reassignment rejected
Assigned advisor / unassigned advisor	Assigned permitted profile; unassigned denied
Current delegation / revoked delegation	Current scope works; revoked scope fails
Current reservation / expired reservation	Current can confirm; expired cannot
Two concurrent claims on one bed	Only one succeeds

Concurrent reservation tests require separate database sessions or an integration test harness. A single sequential pgTAP transaction is not a complete concurrency test.

PostgreSQL superusers and `BYPASSRLS` roles bypass row security. Run behavioral assertions under nonprivileged runtime roles; use administrative privileges only for fixture setup. [1]

3. Default-deny RLS structure

Baseline

```
ALTER TABLE semester.housing_allocations
    ENABLE ROW LEVEL SECURITY;

ALTER TABLE semester.housing_allocations
    FORCE ROW LEVEL SECURITY;

REVOKE ALL ON semester.housing_allocations FROM PUBLIC;
```

With RLS enabled and no applicable policy, PostgreSQL uses default deny: ordinary queries cannot see or modify rows. Table privileges and RLS are separate layers.[1]

Policy architecture

Use:

```
Table privilege
      AND
At least one applicable permissive policy
      AND
Every applicable restrictive policy
```

Permissive policies combine with `OR`; restrictive policies combine with `AND`. Adding a permissive "deny" policy does not override another permissive allow policy.[1]

Example policy structure

The following is a template, not directly runnable. It assumes:

- A nonprivileged `semester_runtime` database role.
- Trusted identity-resolution functions.
- A reviewed scope-checking function.
- Least-privilege grants to those functions.

```
-- Applies a tenant boundary to every allowed operation.
CREATE POLICY housing_tenant_boundary
ON semester.housing_allocations
AS RESTRICTIVE
FOR ALL
TO semester_runtime
USING (
    tenant_id = semester_auth.current_tenant_id()
)
WITH CHECK (
    tenant_id = semester_auth.current_tenant_id()
);
```

```
-- Students may read their own allocation.
CREATE POLICY housing_read_own
ON semester.housing_allocations
AS PERMISSIVE
FOR SELECT
TO semester_runtime
USING (
    person_id = semester_auth.current_person_id()
    AND semester_auth.has_active_membership(tenant_id)
);


-- Staff may read only their assigned operational scope.
CREATE POLICY housing_read_staff_scope
ON semester.housing_allocations
AS PERMISSIVE
FOR SELECT
TO semester_runtime
USING (
    semester_auth.can_read_housing_allocation(
        tenant_id,
        id
    )
);
```

Do not create generic student `UPDATE` access to allocations. Students should request changes through authorized commands.

Trusted identity requirements

`current_tenant_id()` and `current_person_id()` must derive from verified authentication and active memberships.

They must not blindly trust:

```
current_setting('app.tenant_id')
```

if the caller can freely set that value.

For pooled application connections, use a verified gateway/session mechanism or another reviewed identity binding. Do not expose arbitrary SQL execution to users.

Additional boundaries

- RLS controls rows, not every field. Use column grants, safe projections, or APIs for field minimization.
- Search and files need matching controls.
- Scope helpers must not recursively query the protected table without a carefully designed access path.

- Review `SECURITY DEFINER` functions, owners, search paths, and grants.
- Apply current grant checks for sensitive writes.
- Restrict `TRUNCATE`, schema changes, and privileged role membership.

PostgreSQL also warns that policy lookups involving other tables can create race conditions. Authorization revocation and consequential mutations need transaction-aware design. [1]

4. LTI token audit check

Audit contract

The monitor records results from the validator. It must not decode a token and label its claims "verified."

```
Trusted registration
       ↓
Cryptographic validation
       ↓
State/nonce/browser binding
       ↓
Deployment/message/resource checks
       ↓
Atomic replay prevention
       ↓
Local mapping and authorization
       ↓
Sanitized audit event
```

LTI resource-link messages require type, version, deployment, target, resource link, and roles. Context is optional; anonymous launches may omit user identity. [2]

TypeScript audit builder

```
type CheckResult =
  | "pass"
  | "fail"
  | "not_run"
  | "not_applicable";

type LtiCheck =
  | "signature"
  | "issuer"
  | "audience"
  | "authorized_party"
  | "time_window"
  | "state"
  | "nonce"
  | "browser_binding"
  | "deployment"
  | "message_type"
  | "version"
```

```
  | "target_uri"
  | "resource_link"
  | "roles_shape"
  | "roles_vocabulary"
  | "subject"
  | "context"
  | "mentor_scope"
  | "replay";

type ValidationEvidence = {
  attemptId: string;
  requestId: string;

  registrationId?: string;
  deploymentId?: string;

  // Tenant is assigned only from a trusted local mapping.
  trustedTenantId?: string;

  checks: Record<LtiCheck, CheckResult>;

  failureCode?: string;
  subjectTokenId?: string;
  localSectionId?: string;
  localResourceId?: string;

  mappedPermissions: readonly string[];
  authorizedPermissions: readonly string[];

  recognizedCustomKeys: readonly string[];
};

export function buildLtiAudit(
  evidence: ValidationEvidence,
) {
  const required: LtiCheck[] = [
    "signature",
    "issuer",
    "audience",
    "authorized_party",
    "time_window",
    "state",
    "nonce",
    "browser_binding",
    "deployment",
    "message_type",
    "version",
    "target_uri",
    "resource_link",
    "roles_shape",
    "roles_vocabulary",
    "subject",
    "replay",
  ];

  const failed = Object.entries(evidence.checks)
```

```
    .filter(([, result]) => result === "fail")
    .map(([check]) => check);

  const incomplete = required.filter(
    check => evidence.checks[check] !== "pass",
  );

  // Identified resource-link handler.
  const accepted =
    failed.length === 0 &&
    incomplete.length === 0 &&
    evidence.trustedTenantId !== undefined &&
    evidence.deploymentId !== undefined;

  return {
    eventType: "lti.resource_launch.validation",
    attemptId: evidence.attemptId,
    requestId: evidence.requestId,

    tenantId: evidence.trustedTenantId ?? null,
    registrationId: evidence.registrationId ?? null,
    deploymentId: evidence.deploymentId ?? null,

    outcome: accepted ? "allow" : "deny",
    failureCode: evidence.failureCode ?? null,

    checks: evidence.checks,
    failedChecks: failed,
    incompleteChecks: incomplete,

    subjectTokenId: evidence.subjectTokenId ?? null,
    sectionId: evidence.localSectionId ?? null,
    resourceId: evidence.localResourceId ?? null,

    mappedPermissions: [...evidence.mappedPermissions],
    authorizedPermissions: [
      ...evidence.authorizedPermissions,
    ],

    customKeys: [...evidence.recognizedCustomKeys],
  };
}
```

This builds an audit record; it does not perform cryptographic verification. Only the trusted validator should construct `ValidationEvidence`.

Corrections to the earlier validator

Issue	Required correction
Subject validation	Enforce the LTI/OIDC maximum 255 ASCII characters, not only string length
Nonempty roles array	Require at least one role from the defined vocabulary
Unknown roles	Ignore for authorization; do not promote them

Issue	Required correction
Optional context	If absent, do not grant course-scoped capabilities without an approved mapping
Anonymous launch	Use a separate supported anonymous handler; never manufacture a person
Mentor scope	Validate IDs and Mentor role; apply local disclosure rules
Failed issuer/signature	Do not assign tenant from unverified claims
Audit persistence failure	Follow a defined fail-closed or bounded durable-fallback policy for sensitive operations

The core specification explicitly requires ASCII-limited `sub`, deployment, resource-link, and context identifiers. A nonempty roles claim must contain at least one role from its standard vocabularies.[2]

Pre-authentication failures without a trusted tenant belong in a restricted platform-security log, not a guessed tenant's student-record audit table.

Never store raw JWTs, bearer tokens, private keys, names, or email in the validation event.

5. Retention safeguard review

Verdict

The earlier safeguard is directionally correct but not production-ready. Its strongest features are fail-closed retirement review, explicit institutional holds, and checks for outstanding inspection requests.

Its main weaknesses are RLS visibility, concurrent hold creation, reliance on mutable review flags, and incomplete disposition auditing.

Review findings

Finding	Risk	Required change
Missing review blocks deletion	Good conservative behavior	Keep
Review expiration blocks deletion	Good conservative behavior	Keep
Records-retained flag blocks deletion	Correct dependency	Require authoritative retirement evidence
Default hold is true	Conservative	Restrict who may clear it
`SECURITY INVOKER` queries requests	Caller may not see all applicable holds	Dedicated disposition role with complete scoped visibility
`FOR UPDATE` locks review row	Does not automatically lock newly inserted requests	Shared subject-lock protocol
Status `closed` treated as resolved	Could hide an unresolved inspection request	Closure must require verified resolution
Trigger handles `DELETE`	Does not handle `TRUNCATE` or privileged bypass	Revoke privileges and monitor privileged operations

Finding	Risk	Required change
No disposition evidence	Deletion may remove the only local trace	Persist approved disposition manifest
Review state can change	Approval provenance can be lost	Append-only review decisions plus current-state projection

Shared locking protocol

For each subject, create a stable retention-control row.

Both request intake and disposition must acquire the same lock:

```
Inspection request intake:
  Lock subject retention-control row
  Record request
  Activate preservation hold
  Commit

Disclosure disposition:
  Lock same subject retention-control row
  Recheck education-record retention and all holds
  Verify approved disposition
  Record disposition evidence
  Delete eligible records
  Commit
```

This prevents the two database workflows from independently deciding that the other is absent.

If intake occurs through external email, paper, or another university system, provide an immediate preservation process and synchronization path. Database locking cannot protect against a request the institution received but has not recorded.

Statutory boundaries

- Required disclosure records remain as long as the student's education records are maintained.
- Education records cannot be destroyed while an inspection/review request is outstanding.
- Inspection responses are due within a reasonable period, no more than 45 days after receipt.
- Recordkeeping exceptions must be classified separately from operational access logging. [3]

Release criteria

Do not activate automated disposition until tests prove:

- Missing, expired, or unapproved review rejects deletion.
- Records-retained and hold states reject deletion.

- Outstanding inspections reject deletion.
- Ordinary application roles cannot delete or truncate.
- Concurrent intake/disposition uses the same lock.
- Required records remain available for institutional inspection.
- Approved disposition produces a durable manifest.
- External copies, files, backup lifecycle, and institutional authority are covered by the retention procedure.

The production target is not “a trigger exists.” It is a verified chain from institutional retention authority to safe database enforcement, auditable disposition, and recoverable operations.

⁂

1. https://www.postgresql.org/docs/current/ddl-rowsecurity.html
2. https://www.imsglobal.org/spec/lti/v1p3
3. https://studentprivacy.ed.gov/ferpa